

ITEC634 – Week 12 Lab 6 Report

NAME - EMMANUEL ORUNTA

STUDENT ID – S00403657

Week 12 Lab 6 – Cross-Platform Weather Application using .NET MAUI (VS Code + Android Emulator)

1. Introduction

This lab focused on setting up a complete .NET MAUI development environment and building a cross-platform application using Visual Studio Code. The required baseline objective was to create and run a default Hello World MAUI app. The project was then extended into a Weather Forecast application that allows users to enter a city name and fetch live weather data from the OpenWeatherMap REST API. The work included environment setup, project creation using the .NET CLI, emulator configuration, Windows desktop execution, API integration, and debugging build/runtime issues.

2. Objectives of the Lab

The main objectives of this lab were:

- To configure a working .NET MAUI development environment
- To set up an Android emulator for mobile testing
- To create a blank MAUI project using **.NET CLI (dotnet new maui)**
- To compile and run the baseline Hello World app
- To extend the app into a Weather Forecast application (recommended FR-3)
- To fetch and display weather details (city, temperature, condition) from an external API
- To test the application on at least two targets (Android emulator and Windows desktop)

3. Implementation Process

Environment Setup (VS Code + .NET MAUI + Android Tools)

The development environment was prepared using VS Code and .NET CLI.

- Installed/verified .NET SDK
- Verified MAUI workloads:
 - **dotnet workload list**
- Used Android Studio only for emulator/SDK management:
 - Device Manager for creating and launching emulator

Project Creation and Hello World (FR-2)

The MAUI project was created from terminal using CLI:

```
dotnet new maui -n Lab6App
```

```
cd Lab6App
```

```
dotnet build
```

This satisfied the requirement to create a blank MAUI project using **.NET CLI**.

Weather App Implementation (FR-3 Recommended)

The default app was extended into a Weather Forecast app.

Implemented components:

- **UI (MainPage.xaml):**
 - City input field (Entry)
 - Search button (Button)
 - Result display (Label)
 - Loading indicator (ActivityIndicator)
- **Logic (MainPage.xaml.cs):**

- HTTP call using HttpClient
- OpenWeatherMap endpoint integration
- JSON deserialization (System.Text.Json)
- Display of:
 - City name
 - Temperature (°C)
 - Weather condition + description
- Error handling for:
 - Empty city input
 - Invalid API key (401)
 - City not found (404)

Build and Run (Android + Windows)

Commands used during build/run validation:

`dotnet build`

`dotnet build -t:Run -f net10.0-android`

`dotnet build -t:Run -f net10.0-windows10.0.19041.0`

Android testing used a running emulator from Android Studio Device Manager.

Windows testing launched the app directly as a desktop app window.

4. Key Features of the Application

- Cross-platform MAUI app scaffolded with .NET CLI
- Functional weather search interface
- API-driven weather retrieval using OpenWeatherMap
- User input handling and validation
- API error handling (invalid key, missing city, not found city)
- Successful build and execution workflow in VS Code
- Testing performed on:

Android emulator

Windows desktop target

5. Challenges Encountered and Solutions

During implementation, several issues were encountered and resolved:

- **Target framework mismatch (net8 vs net10)**
 - **Issue:** Run command used **net8.0-android** while project output was **net10.0-android**.
 - **Solution:** Used matching framework from **.csproj (net10.0-android)**.
- **No device available (XA0010)**
 - **Issue:** Android run failed because emulator was not running.
 - **Solution:** Started emulator in Android Studio Device Manager and verified device availability before run.
- **Project file not found (MSB1003/MSB1009)**
 - **Issue:** Terminal was in wrong folder after reopening VS Code.
 - **Solution:** Navigated to folder containing **.csproj** and/or passed correct project path.
- **Windows file lock packaging errors (MSB3936)**
 - **Issue:** Build failed because files in **obj** were locked by other processes.
 - **Solution:** Stopped active processes, cleaned **bin/obj**, rebuilt; used clean run sequence.
- **Weather fields appearing empty**
 - **Issue:** API response parsed but displayed incomplete values.
 - **Solution:** Improved JSON mapping/deserialization and added robust null-safe rendering and validation.

6. What I Learned

This lab strengthened my understanding of full MAUI workflow from setup to deployment across platforms.

I learned how to:

- Configure and verify MAUI workloads in CLI

- Use Android Studio for emulator setup while coding in VS Code
- Build/run MAUI for both Android and Windows targets
- Integrate a third-party REST API in C#
- Handle common .NET build issues (framework mismatches, path errors, file locks)
- Implement practical UI + networking + error handling in a cross-platform app

7. Use of Artificial Intelligence (AI)

AI tools were used to support completion of the lab requirements, especially for troubleshooting and code refinement.

AI assistance included:

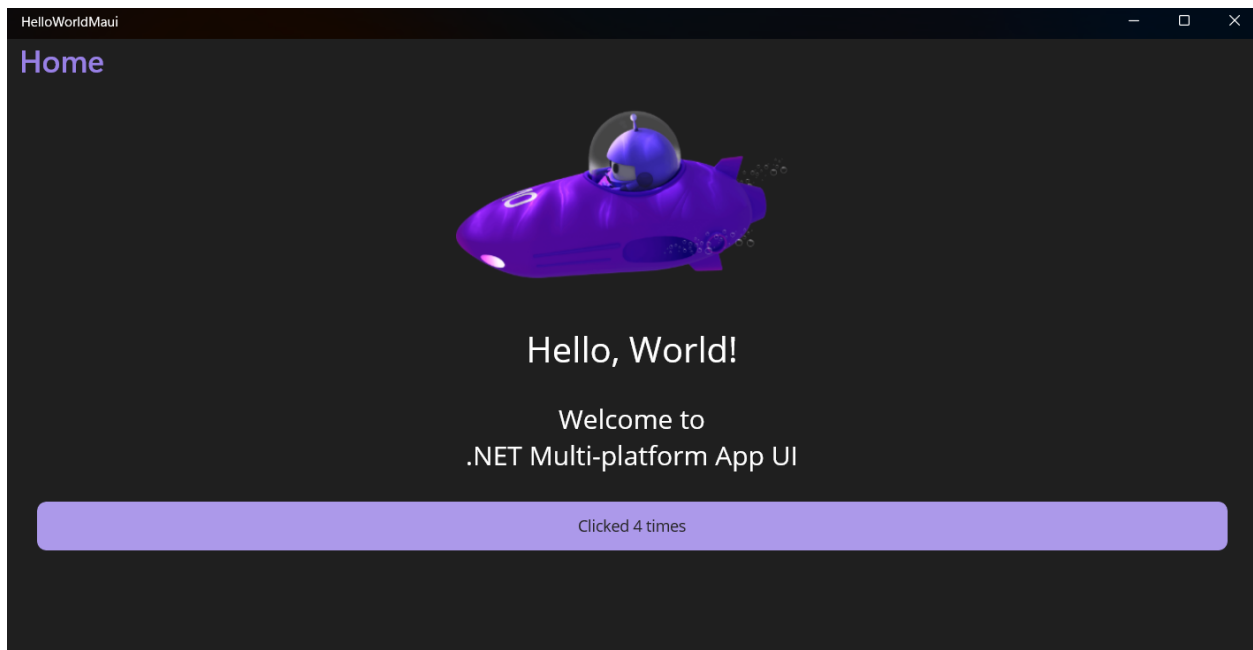
- Step-by-step VS Code setup and run commands
- Weather app code generation and correction
- JSON parsing and API integration guidance
- Debugging help for emulator/device/build errors
- Report structure and evidence checklist preparation

Summary of prompts used:

- How to generate OpenWeatherMap API key
- How to run MAUI on Android emulator and Windows desktop
- How to fix errors (**MSB1003**, **XA0010**, **MSB3936**)
- How to satisfy FR-2 with **.NET CLI**
- What screenshot evidence to capture for grading

Summary of AI responses:

AI provided clear implementation steps, corrected command usage, and troubleshooting flow that helped complete environment setup, app execution, and assignment documentation efficiently.

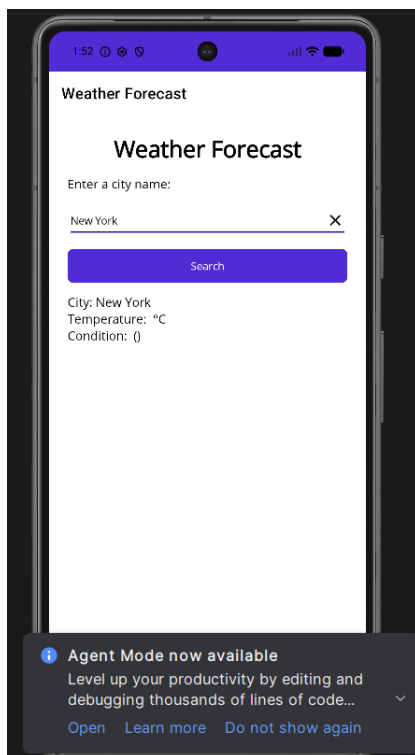
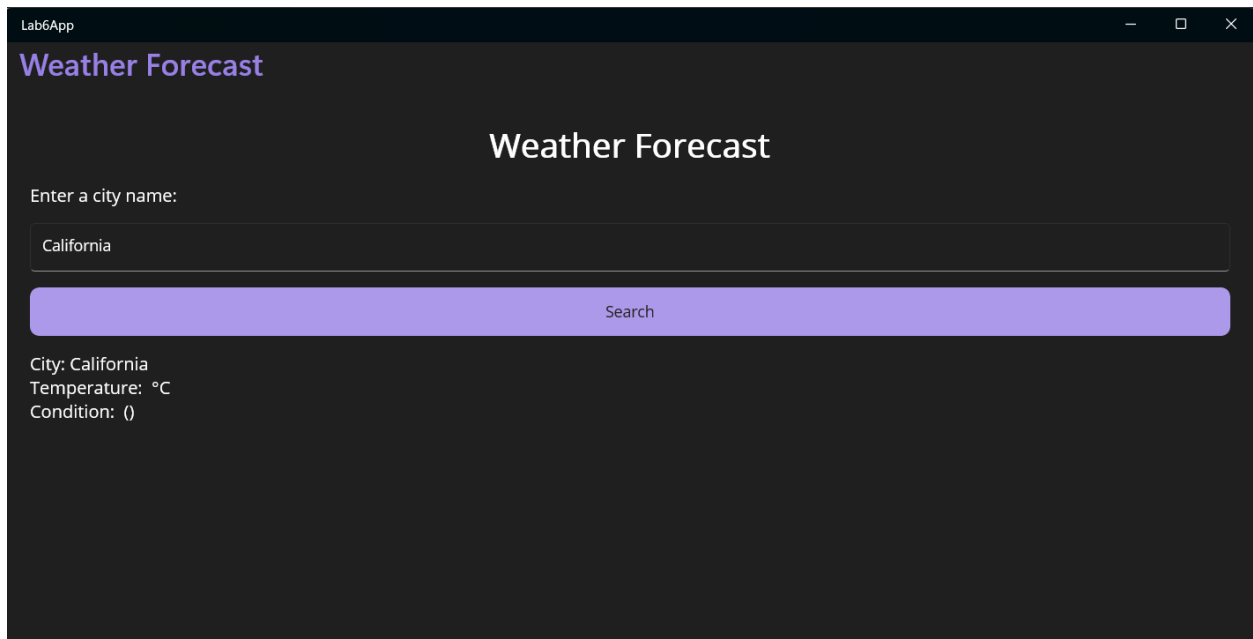


```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
dotnet
PS C:\Users\Emma\Downloads\HelloWorldMaui> dotnet new maui -n HelloWorldMaui
>> cd .\HelloWorldMaui
>> dotnet build
The template ".NET MAUI App" was created successfully.

Processing post-creation actions...
Restoring C:\Users\Emma\Downloads\HelloWorldMaui\HelloWorldMaui\HelloWorldMaui.csproj:
Restore succeeded.

Restore complete (1.6s)
  HelloWorldMaui net10.0-ios iosimulator-x64 succeeded (33.7s) -> bin\Debug\net10.0-ios\iosimulator-x64\HelloWorldMaui.dll
  HelloWorldMaui net10.0-maccatalyst maccatalyst-x64 succeeded (34.5s) -> bin\Debug\net10.0-maccatalyst\maccatalyst-x64\HelloWorldMaui.dll
  HelloWorldMaui net10.0-windows10.0.19041.0 win-x64 succeeded (88.6s) -> bin\Debug\net10.0-windows10.0.19041.0\win-x64\HelloWorldMaui.dll
  HelloWorldMaui net10.0-android succeeded (226.8s) -> bin\Debug\net10.0-android\HelloWorldMaui.dll

Build succeeded in 233.1s
PS C:\Users\Emma\Downloads\HelloWorldMaui\HelloWorldMaui> dotnet build -t:Run -f net10.0-windows10.0.19041.0
  HelloWorldMaui net10.0-windows10.0.19041.0 win-x64 Run (1000.2s)
```



Screenshots of the Application in Android Studio

8. Conclusion

In conclusion, this lab achieved the required outcomes:

- MAUI environment configuration completed
- Blank Hello World MAUI app created using **.NET CLI**
- Cross-platform execution validated
- Weather Forecast extension implemented using external REST API
- Testing evidence collected on Android emulator and Windows desktop

Overall, the lab provided strong practical experience in cross-platform app development with .NET MAUI, API integration, and real-world debugging in VS Code.